

Task 10.5

Practical Regression Using Machine-Learning Libraries

Name: Samaira Singh

Registration Number: 251090051002

Domain: Software and ML

Repository: github.com/samaira-s/Synergy_TP

1. Introduction and Problem Statement

This report documents the design, implementation, and evaluation of a regression pipeline built with scikit-learn to predict a continuous Temperature value from five sensor readings. The work extends the from-scratch linear/logistic regression foundation built in Task 9.5 into a full library-based workflow covering baseline comparison, multiple model families, evaluation, error analysis, and model reuse through a saved pipeline and a standalone inference script.

The objective was not only to train a regression model, but to determine how much genuine predictive signal the available sensor features carry for Temperature, and to build a reproducible, reusable pipeline around whatever the best-performing model turned out to be.

2. Dataset Description and Target Selection

The dataset (Data.csv) contains 3,457 rows and 6 numeric columns with no missing values in any column. Each row represents a single instant of sensor readings alongside the corresponding ambient Temperature at that instant.

- Target variable: Temperature — a continuous value ranging from approximately 19°C to 32°C, with a broad, fairly even distribution and no strong skew (see Figure 1).
- Feature set: Sensor1 through Sensor5 — all five features are confined to a 0–1 range with means near 0.5, suggesting they are already normalized sensor outputs rather than raw physical units.
- No missing values were found in any column, so no imputation was required.

A useful prediction in this context means estimating the ambient Temperature accurately enough from sensor readings alone that the model could substitute for a direct temperature reading, or flag when sensor-inferred temperature diverges from expectation.

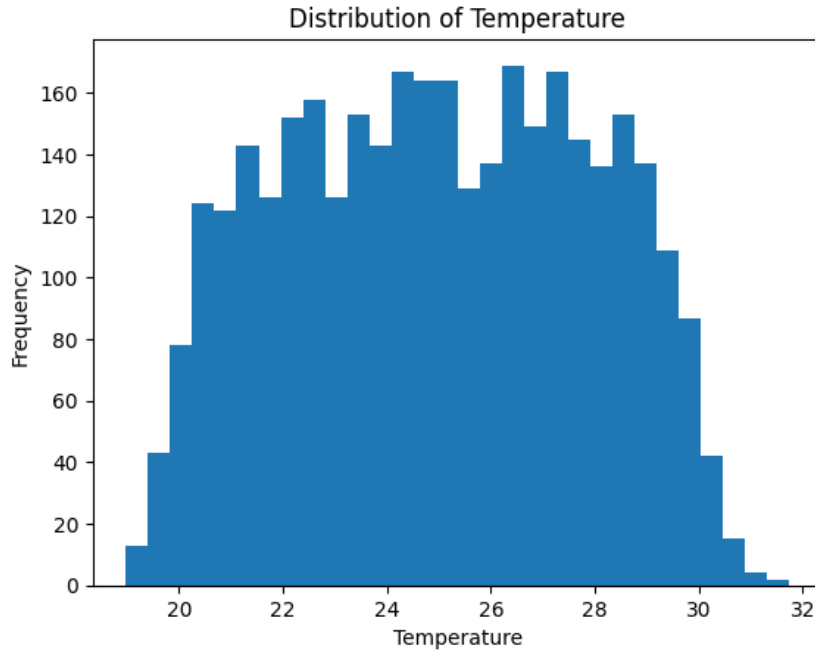


Figure 1. Distribution of the Temperature target variable.

3. Data Preparation and Experimental Approach

3.1 Feature Relevance Analysis

Before model training, Pearson and Spearman correlations were computed between each sensor and Temperature, and a full correlation matrix was inspected across all features (Figure 2).

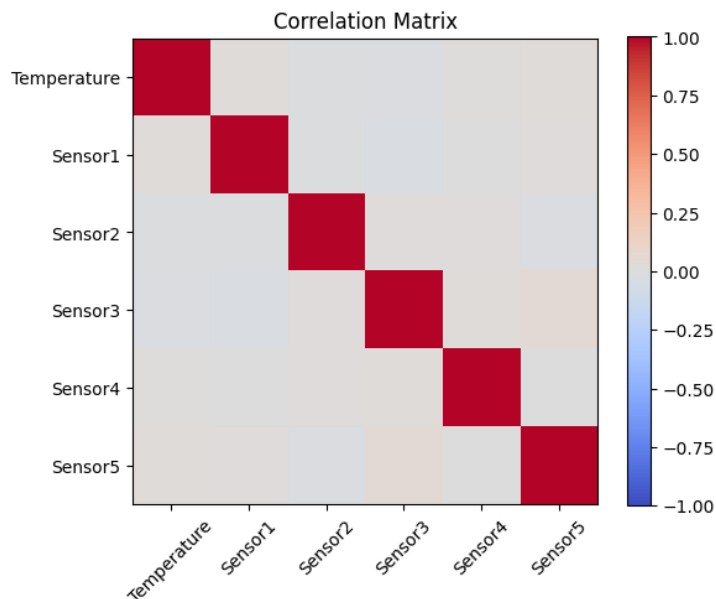


Figure 2. Correlation matrix across Temperature and all five sensors.

Both Pearson and Spearman correlations were near zero for every sensor, and the two measures matched closely. Since Pearson captures linear relationships and Spearman captures monotonic (rank-based) relationships, their agreement here indicates the absence of either kind of pairwise relationship between any single sensor and Temperature — not merely a nonlinear-but-monotonic pattern that Pearson alone would miss. Sensors were also weakly correlated with each other, ruling out multicollinearity as a concern for the linear models.

3.2 Data Splitting

The dataset was split into training, validation, and test sets in an approximately 70/15/15 ratio using a fixed random seed (`random_state=42`) for reproducibility. This produced 2,419 training rows, 519 validation rows, and 519 test rows. All model comparison and selection was performed on the validation set; the test set was reserved and untouched during model development.

3.3 Preprocessing

A scikit-learn Pipeline was used for every model, combining a `StandardScaler` preprocessing step with the estimator. This ensures preprocessing statistics are learned only from training data and applied consistently to validation, test, and any future inference data, and allows the entire fitted pipeline — scaler included — to be saved and reused as a single object.

4. Regression Models Used

- `DummyRegressor (strategy='mean')` — baseline that always predicts the mean Temperature, used as the minimum bar every real model must clear.
- `LinearRegression` — standard ordinary least squares regression.
- `Ridge (alpha=1.0)` — linear regression with L2 regularization to shrink coefficients and guard against instability from correlated features.
- `DecisionTreeRegressor (max_depth=5)` — a single regression tree, depth-limited to reduce overfitting.
- `RandomForestRegressor (n_estimators=200)` — an ensemble of trees; evaluated at two depths (`max_depth=8` and `max_depth=4`) to study the effect of model complexity on generalization.

5. Evaluation Metrics and Model-Comparison Table

Each model was evaluated on the validation set using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 . MAE and RMSE are reported in $^{\circ}\text{C}$; R^2 measures improvement over always predicting the mean (0 = no improvement, negative = worse than the mean).

Model	MAE	RMSE	R^2
Dummy (baseline)	2.4700	2.8869	-0.0001
LinearRegression	2.4760	2.8928	-0.0042
Ridge	2.4760	2.8928	-0.0042
DecisionTree	2.5087	2.9658	-0.0556

RandomForest (final, depth=4)	2.4606	2.8793	0.0051
-------------------------------	--------	--------	--------

No model achieves a meaningfully positive R^2 . LinearRegression, Ridge, and DecisionTree all perform at or slightly below the Dummy baseline. The final RandomForest configuration (max_depth=4) is the only model to beat the baseline on all three metrics, though the margin is small ($R^2 = 0.0051$ versus 0.0000 for the baseline). This is consistent with the near-zero Pearson and Spearman correlations found during data preparation: the available sensor features carry very little linear or nonlinear single-feature signal about Temperature.

6. Prediction Plots and Error Analysis

Actual-versus-predicted and residual plots were generated for the final RandomForest model (Figures 3 and 4).

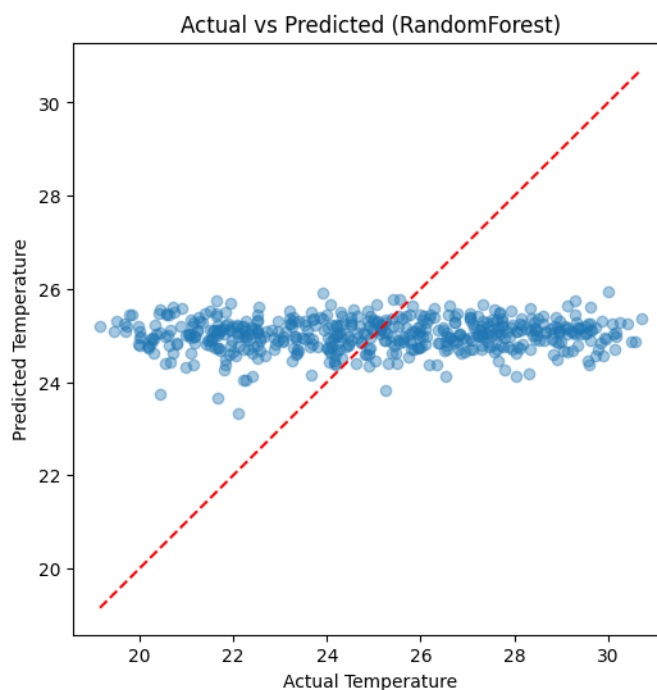


Figure 3. Actual vs. predicted Temperature, RandomForest (validation set).

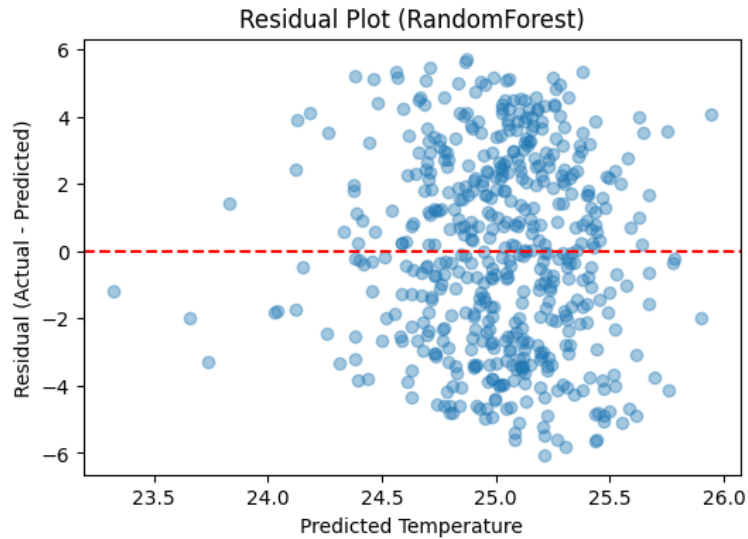


Figure 4. Residual plot, RandomForest (validation set).

The actual-vs-predicted plot shows predictions compressed into a narrow band (roughly 24–26°C) regardless of the true Temperature, which itself spans 19–31°C. This is the visual signature of a model with near-zero R^2 : with little usable signal in the features, the model defaults to predicting close to the overall mean for nearly every input. The residual plot shows a fairly random scatter around zero with no funnel shape or curved trend, indicating no additional structural problem (such as heteroscedasticity) beyond the underlying weak signal.

6.1 High-Error Predictions

The fifteen largest-error validation predictions were inspected directly. The five largest are shown below; the full set of fifteen was exported to `high_error_predictions.csv`.

Actual (°C)	Predicted (°C)	Abs Error
19.16	25.21	6.05
19.51	25.31	5.79
30.58	24.87	5.71
30.51	24.87	5.64
19.80	25.43	5.64

Every one of the fifteen largest errors occurs at the extremes of the Temperature range — either close to 19–20°C or close to 29.5–30.7°C — with no worst-error case near the middle of the distribution. This indicates the errors are not random noise but a structural limitation: because the model predicts close to the mean for almost every input, it is systematically unable to reach either tail of the Temperature range, producing large misses whenever the true value is far from average.

7. Final Model Selection and Justification

RandomForestRegressor was selected as the final model, but only after correcting an overfitting issue found during development.

An initial RandomForest with `max_depth=8` showed a substantial gap between training R^2 (0.2322) and validation R^2 (0.0017), indicating the model was fitting patterns in the training data — likely noise, given how weak the underlying signal is — that did not generalize. Reducing `max_depth` to 4 nearly closed this gap (train R^2 0.0540, val R^2 0.0051) while slightly improving validation performance, evidence that a simpler model generalizes at least as well, if not better, on this dataset.

RandomForest Variant	Train R^2	Val R^2
<code>max_depth = 8</code>	0.2322	0.0017
<code>max_depth = 4 (final)</code>	0.0540	0.0051

Final model: RandomForestRegressor(`n_estimators=200`, `max_depth=4`, `random_state=42`). This was chosen over LinearRegression and Ridge because it is the only model to beat the Dummy baseline on all three metrics, and over the deeper RandomForest variant because it generalizes without loss of validation performance. It is important to be transparent that this improvement over baseline is small in absolute terms; the honest conclusion is that RandomForest is the best available option among those tested, not that it achieves strong predictive accuracy in absolute terms.

8. Model Saving and Inference Workflow

The final fitted pipeline (StandardScaler + RandomForestRegressor, `max_depth=4`) was serialized using joblib to `final_regression_pipeline.joblib`. Saving the complete pipeline, rather than the model alone, ensures the scaling step travels with the model and is applied identically at inference time.

A separate script, `predict.py`, was written to load this saved pipeline independently of the training notebook. It supports two modes: predicting from a single hardcoded record, and predicting from an arbitrary input CSV of new sensor readings, writing results to `predictions_output.csv`. Both modes were tested from a standalone terminal session and produced sensible outputs consistent with the model's overall tendency to predict near the mean.

To confirm reload correctness, predictions from the freshly reloaded pipeline were compared against predictions from the original in-memory pipeline on the validation set using `numpy.allclose()`, which returned True — confirming the saved and reloaded pipeline produces identical results to the original.

9. Limitations and Possible Improvements

- The available sensor features show near-zero linear (Pearson) and monotonic (Spearman) correlation with Temperature, both individually and pairwise with each other. This is the primary limitation: no model tested, regardless of complexity, could extract strong predictive signal from these features alone.

- Because even training-set R^2 remained low for the depth-4 RandomForest (0.054), the issue is better characterized as insufficient signal in the features rather than classic overfitting, though overfitting was observed and corrected at higher tree depth.
- The model systematically underpredicts at the high end of the Temperature range and overpredicts at the low end, since it defaults toward the mean; it is least reliable precisely for the most extreme (and often most operationally relevant) readings.
- Possible improvements: engineering additional features (e.g. interaction terms or time-based features, if timestamps are available), collecting additional sensors more directly related to temperature, or exploring whether a combination of sensors — rather than any single one — carries interaction-based signal that pairwise correlation cannot detect.
- Hyperparameter tuning beyond the two RandomForest depths tested (e.g. a full grid or randomized search across depth, number of estimators, and minimum samples per leaf) could be explored, though given the weak baseline correlations, large gains are unlikely without better features.

10. Conclusion

This task implemented a complete scikit-learn regression workflow — baseline comparison, multiple linear and tree-based models, systematic evaluation, error analysis, and a reusable saved pipeline with an independent inference script. The key finding is that the provided sensor features carry very little predictive signal for Temperature: no model meaningfully outperforms simply predicting the mean, and the best-performing model (RandomForest, `max_depth=4`) does so only marginally. Rather than a failure of the pipeline, this reflects a genuine property of the dataset, established through Pearson and Spearman correlation analysis before any model was trained and confirmed again by every model's evaluation metrics. The full pipeline — including overfitting diagnosis and correction, error-tail analysis, and a working save/reload/inference cycle — is complete and reproducible.